



INF398 - Introducción al aprendizaje automático

Recommender systems

Alejandro Veloz

Outline

From embeddings to recommendations

Feedback: explicit vs implicit

Collaborative filtering and matrix factorization

Deep recommender architectures

Ranking losses for implicit feedback

Evaluation, deployment, and ethics

From embeddings to recommendations

Recall: what is an embedding?

An embedding maps a discrete entity to a trainable vector:

$$s \in V \longrightarrow z_s \in \mathbb{R}^d$$

- words had embeddings,
- images had embeddings,
- multimodal models aligned embeddings.

In this lecture: users and items will also have embeddings.

Recommendation as matching

In a recommender system, we want to score a user-item pair:

$$\text{score}(u, i)$$

Examples:

- Will this user watch this movie?
- Will this patient click this health article?
- Will this listener replay this song?
- Which item should appear at rank 1?

Recommendation is a matching problem between user representations and item representations.

Users and items as symbols

User ids and item ids are categorical variables:

$$\text{user_id} \in \{1, \dots, N_u\}, \quad \text{item_id} \in \{1, \dots, N_i\}$$

We can learn:

$$p_u \in \mathbb{R}^d, \quad q_i \in \mathbb{R}^d$$

where p_u is the user embedding and q_i is the item embedding.

Basic scoring model

The simplest embedding recommender scores compatibility with a dot product:

$$\hat{r}_{ui} = p_u^\top q_i$$

- large dot product: user and item are compatible,
- small dot product: weak match,
- negative dot product: mismatch, if embeddings are unconstrained.

Dot products are useful because they are fast for ranking millions of items.

A slightly better rating model

Ratings often need baseline terms:

$$\hat{r}_{ui} = \mu + b_u + c_i + p_u^\top q_i$$

- μ : global average rating
- b_u : user bias
- c_i : item bias
- $p_u^\top q_i$: personalized interaction

Embeddings model what is specific to the user-item match; biases model general tendencies.

Feedback and problem formulation

Recommender systems

Recommend contents and products:

- movies on Netflix and YouTube,
- weekly playlists and related artists on Spotify,
- books and products on Amazon,
- “Who to Follow” on social platforms,
- personalized search results,
- personalized ads.

The technical task is usually ranking, not just prediction.

Content-based vs collaborative filtering

Content-based. Uses metadata or features:

- User metadata: age, location, device, language.
- Item metadata: genre, year, author, tags, text, image.

Collaborative filtering. Uses user-item interactions:

- ratings, plays, purchases, likes, clicks, dwell time.

Hybrid systems. Use both interactions and metadata.

Why hybrid systems matter

Collaborative filtering is powerful, but it has a cold-start problem:

- new users have few interactions,
- new items have few interactions.

Metadata can help:

new item features $\rightarrow$ initial item embedding

Collaborative filtering learns from behavior; content features help when behavior is missing.

Explicit feedback

Explicit feedback gives direct preference labels:

- star ratings,
- likes and dislikes,
- votes,
- reviews.

Common task:

$$\hat{r}_{ui} \approx r_{ui}$$

Common metrics: RMSE, MAE.

Implicit feedback

Implicit feedback records behavior, not declared preference:

- clicks,
- plays,
- purchases,
- page views,
- watch time,
- skips,
- dwell time.

Implicit feedback is abundant, but ambiguous.

Why implicit feedback is tricky

A click is not always a preference.

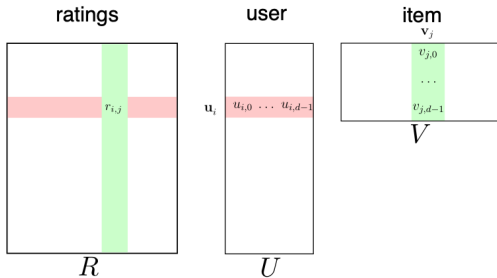
Examples:

- a click with very short dwell time may be negative,
- a skip may be negative,
- repeated listening may be strong positive signal,
- non-clicks can mean dislike, lack of exposure, or bad ranking.

The recommender system changes the data it later observes.

Collaborative filtering

Matrix factorization for collaborative filtering



$$\hat{r}_{ui} = p_u^\top q_i$$

Learn user and item embeddings from observed entries in a sparse interaction matrix.

Matrix factorization objective

For explicit ratings:

$$\min_{P,Q} \sum_{(u,i) \in \Omega} (r_{ui} - p_u^\top q_i)^2 + \lambda \left(\sum_u \|p_u\|_2^2 + \sum_i \|q_i\|_2^2 \right)$$

- Ω : observed user-item pairs
- P : user embedding matrix
- Q : item embedding matrix
- λ : regularization strength

Matrix factorization as embeddings

Matrix factorization is an embedding model:

$$u \rightarrow p_u, \quad i \rightarrow q_i$$

Prediction is geometry:

$$\hat{r}_{ui} = p_u^\top q_i$$

Recommender systems are one of the cleanest applications of embeddings.

What do latent factors mean?

Latent factors may capture dimensions such as:

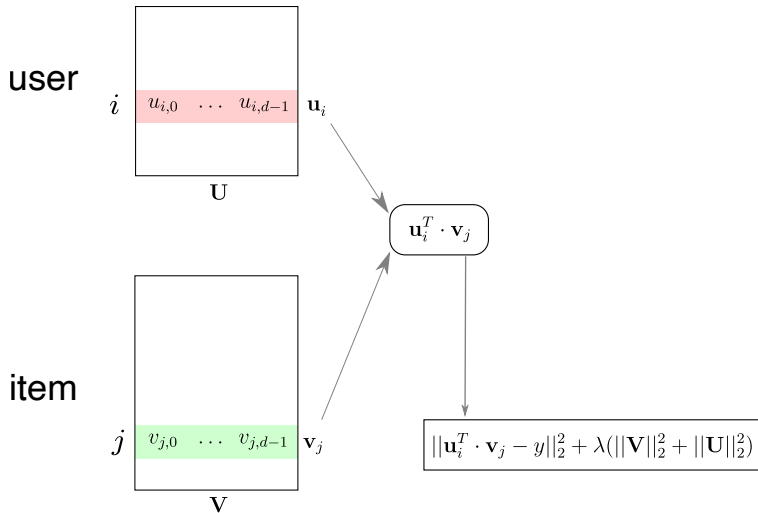
- action vs drama,
- mainstream vs niche,
- short vs long content,
- expensive vs cheap items,
- professional vs casual interests.

But the axes are usually not directly interpretable.

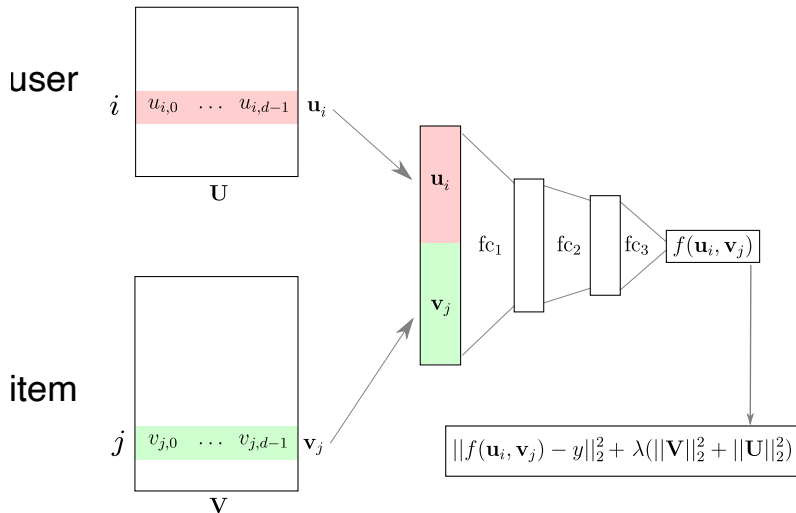
The geometry matters more than individual coordinates.

Deep recommender systems

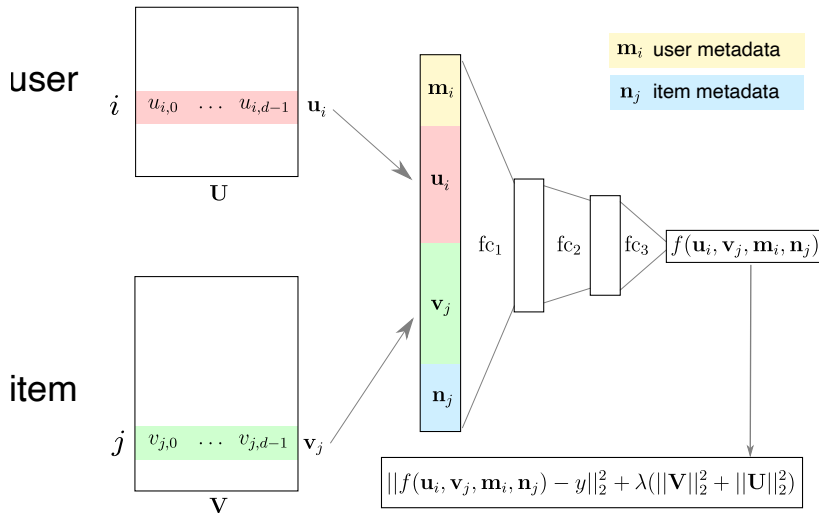
RecSys with explicit feedback



Deep RecSys architecture



Deep RecSys with metadata



Deep scoring model

Instead of only using a dot product, concatenate features:

$$h_{ui} = [p_u, q_i, x_u, x_i]$$

and learn:

$$\hat{r}_{ui} = f_{\theta}(h_{ui})$$

This allows nonlinear interactions between:

- user embeddings, item embeddings, user metadata, and item metadata.

What deep models add

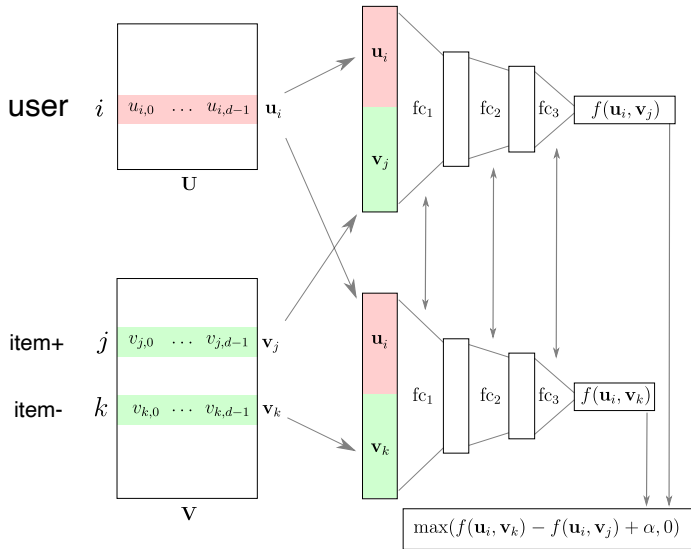
Deep recommender systems can combine:

- embeddings for categorical ids,
- numerical features,
- text embeddings,
- image embeddings,
- context features: time, device, location,
- sequential history.

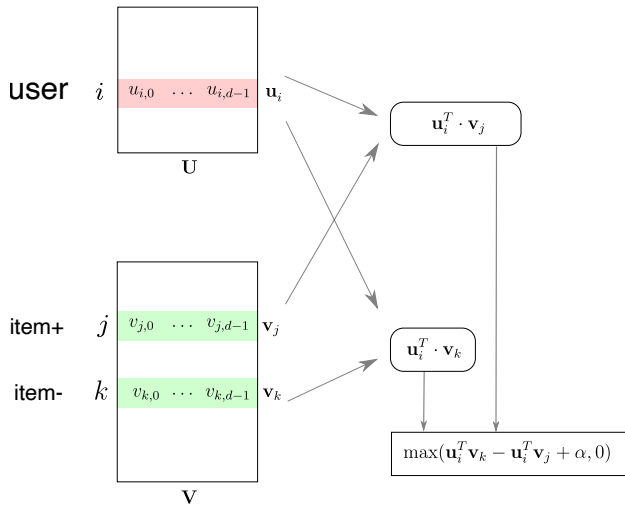
Deep RecSys is not "matrix factorization plus magic"; it is feature learning plus ranking at scale.

Ranking with implicit feedback

Implicit feedback: triplet loss



Deep triplet networks



Training a triplet model

Gather positive pairs (u, i^+) from observed interactions.

For each positive pair:

1. sample a negative item i^- ,
2. compute scores $s(u, i^+)$ and $s(u, i^-)$,
3. update the model so the positive item ranks higher.

One possible loss:

$$\mathcal{L}_{triplet} = \max(0, m - s(u, i^+) + s(u, i^-))$$

BPR ranking loss

Another common implicit-feedback objective is Bayesian Personalized Ranking:

$$\mathcal{L}_{BPR} = -\log \sigma(s(u, i^+) - s(u, i^-))$$

Interpretation:

$$s(u, i^+) > s(u, i^-)$$

With implicit feedback, we often train relative preferences, not absolute ratings.

Negative sampling

Uniform negative sampling:

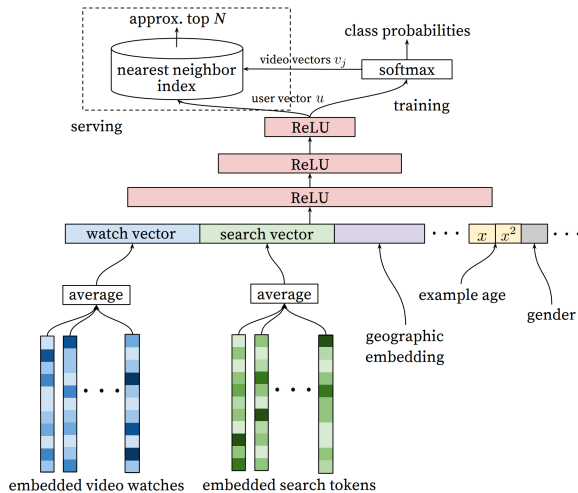
- simple,
- cheap,
- may sample items the user never saw.

Hard negative mining:

- samples plausible confusing negatives,
- often improves ranking,
- can make training unstable if too aggressive.

Negative examples are assumptions, not ground truth.

YouTube neural recommendations



Covington, Adams, and Sargin, 2016.

Candidate generation vs ranking

Large-scale recommenders often have two stages:

Candidate generation

- Retrieve hundreds or thousands of plausible items.
- Optimized for speed and recall.

Ranking

- Score a smaller candidate set more carefully.
- Optimized for ordering and business/user objectives.

Evaluation and deployment

Offline evaluation

For explicit feedback:

- RMSE, MAE.

For ranking:

- precision@k,
- recall@k,
- NDCG@k,
- mean reciprocal rank,
- ROC AUC.

Offline metrics are useful, but they do not fully predict online behavior.

Online evaluation

Online systems care about:

- click-through rate,
- watch time,
- conversion,
- retention,
- satisfaction,
- diversity,
- long-term user value.

The objective you optimize changes the behavior you create.

Distribution shift and feedback loops

Recommenders affect their own training data:

ranking $\rightarrow$ exposure $\rightarrow$ feedback $\rightarrow$ future training data

Consequences:

- Popular items get more exposure.
- Users see narrower content.
- The model may reinforce its own mistakes.
- Missing feedback is not random.

Ethical considerations

Ethical considerations

Possible harms:

- Amplification of existing bias.
- Discriminatory targeting.
- Filter bubbles and polarization.
- Addiction and compulsive use.
- Privacy leakage from behavioral data.
- Unfair exposure for creators or items.

Optimizing engagement can reward extreme or emotionally reactive content.

Designing ethical recommender systems

Practical actions:

- Audit performance across user groups.
- Audit exposure across item/provider groups.
- Avoid sensitive or proxy features when inappropriate.
- Optimize for long-term wellbeing, not only clicks.
- Provide transparency and control.
- Enable independent audits while protecting privacy.

Fair representations

Can we learn representations that remove sensitive information?

Example idea:

$$z = f_{\theta}(x)$$

where z is useful for recommendation but less predictive of a protected attribute.

This connects to adversarial representation learning and fairness-aware modeling.

What this class adds after embeddings

From the embeddings class:

symbol $\rightarrow$ vector

Today:

(user vector, item vector) $\rightarrow$ score or rank

Recommender systems are representation learning systems deployed in feedback loops.

Autoencoders come next

We will learn representations by reconstruction:

$$x \rightarrow z \rightarrow \hat{x}$$

- Embeddings compress users and items.
- Matrix factorization reconstructs missing interactions.
- Autoencoders generalize reconstruction-based representation learning.

Embeddings learn coordinates; recommenders use them to rank; autoencoders learn coordinates by reconstructing data.

Summary

- Recommenders are matching and ranking systems.
- User and item ids can be represented by embeddings.
- Matrix factorization is an embedding model.
- Implicit feedback requires ranking losses and negative sampling.
- Deep RecSys combines ids, metadata, and context.
- Deployment creates feedback loops and ethical risks.
- Autoencoders will extend representation learning through reconstruction.